MIRPUR UNIVERSITY OF SCIENCE AND TECHNOLOGY (MUST)

Web Engineering Semester Project

GROUP NAMES	ROLL NO
Muhammad Qamar	FA23-BSE-046
Muhammad Zeshan	FA23-BSE-050
Waqar Younas	FA23-BSE-066

SUBJECT: Web Engineering

SUBMITTED TO: Dr. Shamila Nasreen

SESSION: 2023-27

DEPARTMENT: SOFTWARE ENGINEERING

Project Title and Introduction

Project Title

Clinic Management System

Project Introduction

The **Clinic Management System (CMS)** is a full-stack web application designed to digitize and streamline clinic operations. It enables:

- Patients to register, login, and book appointments
- Doctors to manage appointment statuses
- Admins to manage doctors, patients, and appointments

The system follows a **RESTful API architecture** with a responsive frontend and a relational MySQL database.

Problem Statement

Many clinics and small healthcare centers still rely on manual processes or basic record-keeping systems to manage patients, doctors, and appointments. This traditional approach often leads to inefficiencies such as misplaced records, scheduling conflicts, long waiting times, data redundancy, and lack of real-time information access. Patients face difficulties in booking appointments conveniently, doctors struggle to manage their schedules effectively, and administrators find it challenging to maintain accurate records of patients, doctors, and appointment histories.
Furthermore, the absence of a centralized system limits transparency, reduces operational efficiency, and increases the risk of human errors.

To address these challenges, there is a need for a centralized, web-based Clinic Management System that automates patient registration, appointment booking, doctor scheduling, and administrative management.
The system should provide role-based access control, secure authentication, real-time data updates, and a user-friendly interface to streamline clinic operations and improve overall service quality.

System Features

Separate Login Portals for

- Patient
- Doctor
- Admin

Secure session management using localStorage

Unauthorized access redirection

Real-time login validation

Patient Registration Features

- Full form validation:
- Name required
- Email regex validation
- Phone (11 digits)
- DOB required
- Gender selection
- Password minimum length

Patient Login

- Secure authentication
- Stores session in localStorage
- Redirects to dashboard on success

Book Appointment

- View available doctors
- Select date and time
- Enter reason for visit
- Submit appointment request
- Success alert and form reset

Patient Dashboard

- View total appointments
- View appointment history
- Cancel appointment (if status = submitted)
- Dynamic updates without page reload

Doctor Features

- Doctor Login
- Secure authentication
- Email format validation
- Session management

Doctor Dashboard

- View assigned appointments
- Update appointment status:
- Cancel

- Mark as Attended
- Real-time dashboard updates
- Appointment statistics display

Admin Features

- Admin Login
- Secure admin authentication
- Session storage & access control

Admin Dashboard

- View total doctors count
- View total patients count
- View all appointments

Delete appointments

- Manage Patients
- View all registered patients
- Delete patient records
- Input validation for deletion
- Dynamic table rendering

Manage Doctors

- Add new doctor
- Delete doctor

- View doctor list
- Full form validation:
- Name
- Email
- Phone (11 digits)
- Specialization
- Password (≥4 characters)

Frontend Features

- Fully responsive design
- Modern UI with glass-style navbar
- Flexbox and Grid layouts
- Sidebar navigation for dashboards
- Hover animations and smooth transitions
- Mobile-friendly tables
- Dynamic DOM rendering
- No page reload system
- Real-time form validation

Backend & API Features

- RESTful API architecture
- CRUD operations:
- Create (Register, Add Doctor, Book Appointment)

- Read (Load dashboards, lists)
- Update (Appointment status)
- Delete (Patients, Doctors, Appointments)
- Role-based access control
- JSON data exchange
- Secure route handling

Technical Features

- Async/Await for API calls
- Fetch API for communication
- LocalStorage for client-side state
- Dynamic table rendering
- Confirmation dialogs before deletion
- Error handling and validation messages

Technology Stack

Frontend

- HTML5
- CSS3 (Responsive Design)
- Vanilla JavaScript
- Fetch API
- LocalStorage (client-side session management)

Backend

- Node.js
- Express.js
- MySQL2
- REST API Architecture

Database

- MySQL
- Relational schema with foreign key constraints
- ENUM-based controlled status system

FrontendDesign Explanation

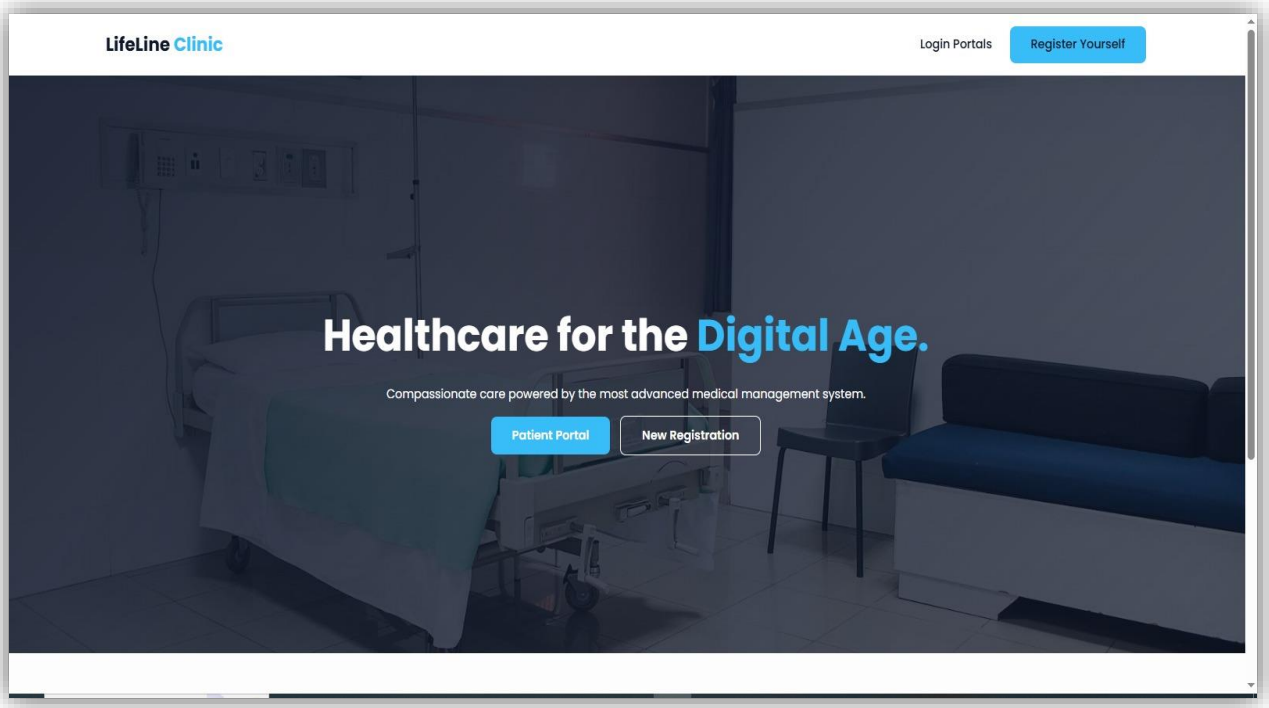
Home Page

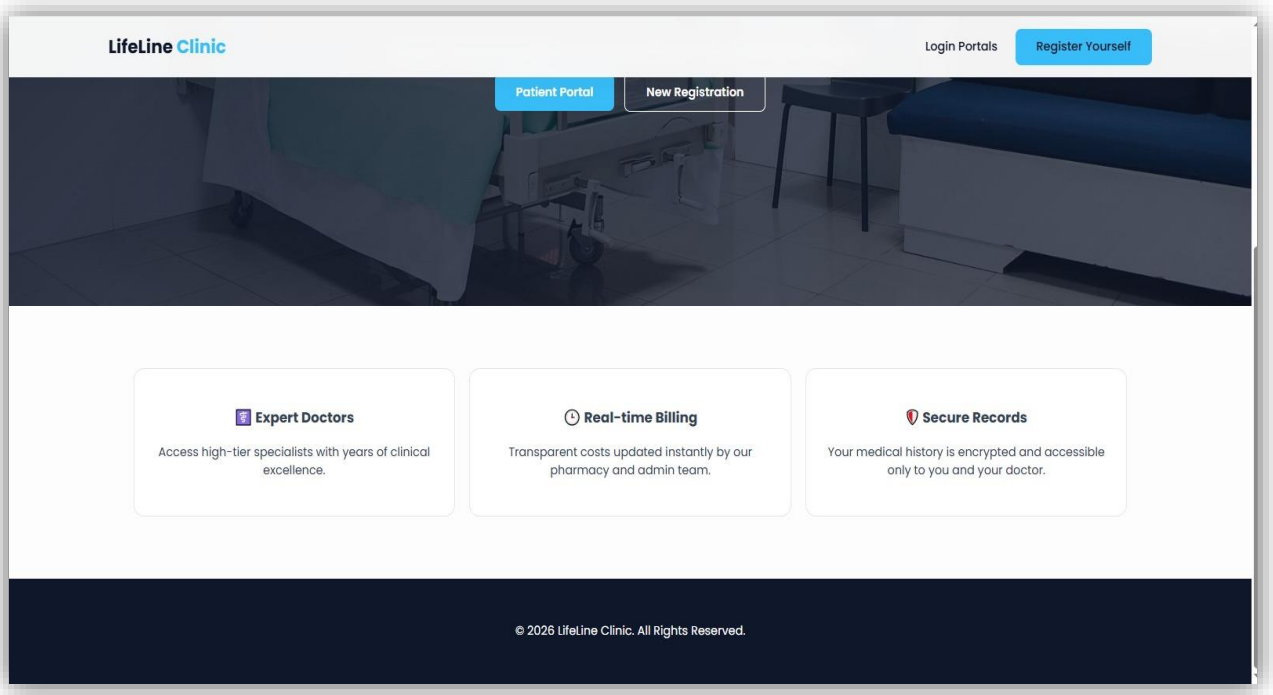
Key Components:

- Navigation bar with logo and action links
- Hero section with headline and CTA buttons
- Features section (3 feature cards)
- Footer

CSS Highlights:

- Google Fonts: Poppins, Barlow Condensed, Google Sans
- Flexbox + Grid layout
- Glass-style transparent navbar
- Fully responsive
- Hover animations on buttons





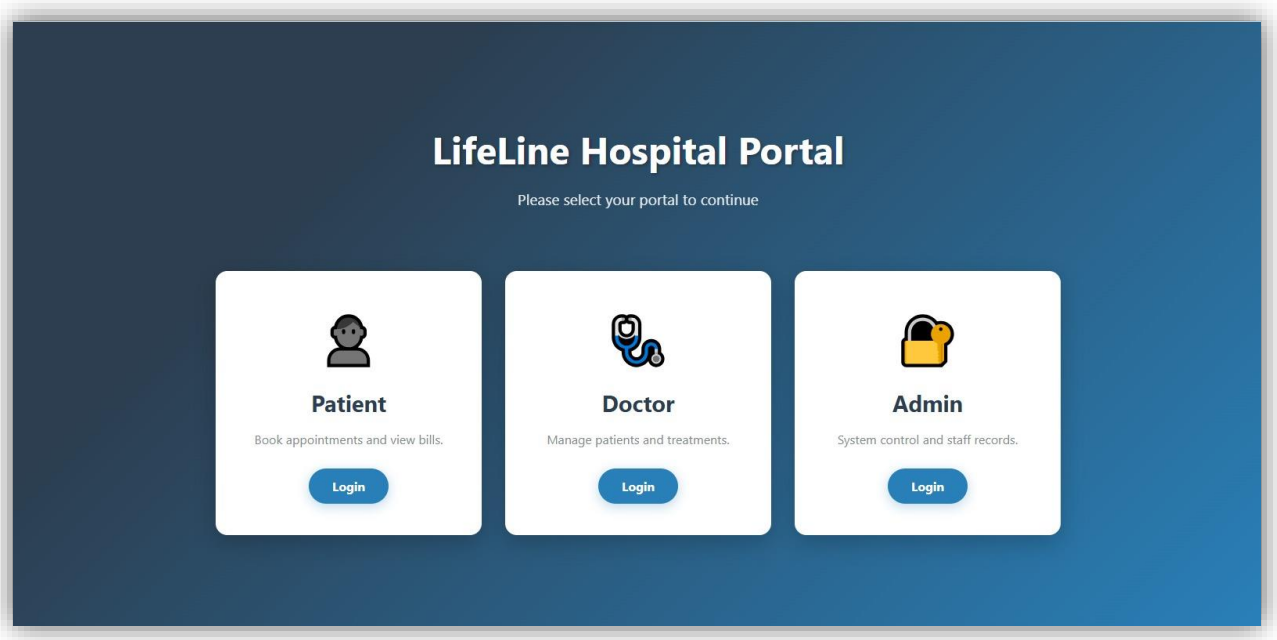
Login Portals Page

Key Components:

- Portal selection cards (Patient / Doctor / Admin)
- Gradient background
- Hover effects
- Responsive stacking on mobile

CSS Highlights:

- Flexbox and card layout
- Gradient backgrounds for modern look
- Responsive design for tablets and mobiles
- Hover animations on cards



Patient Login

Key Components:

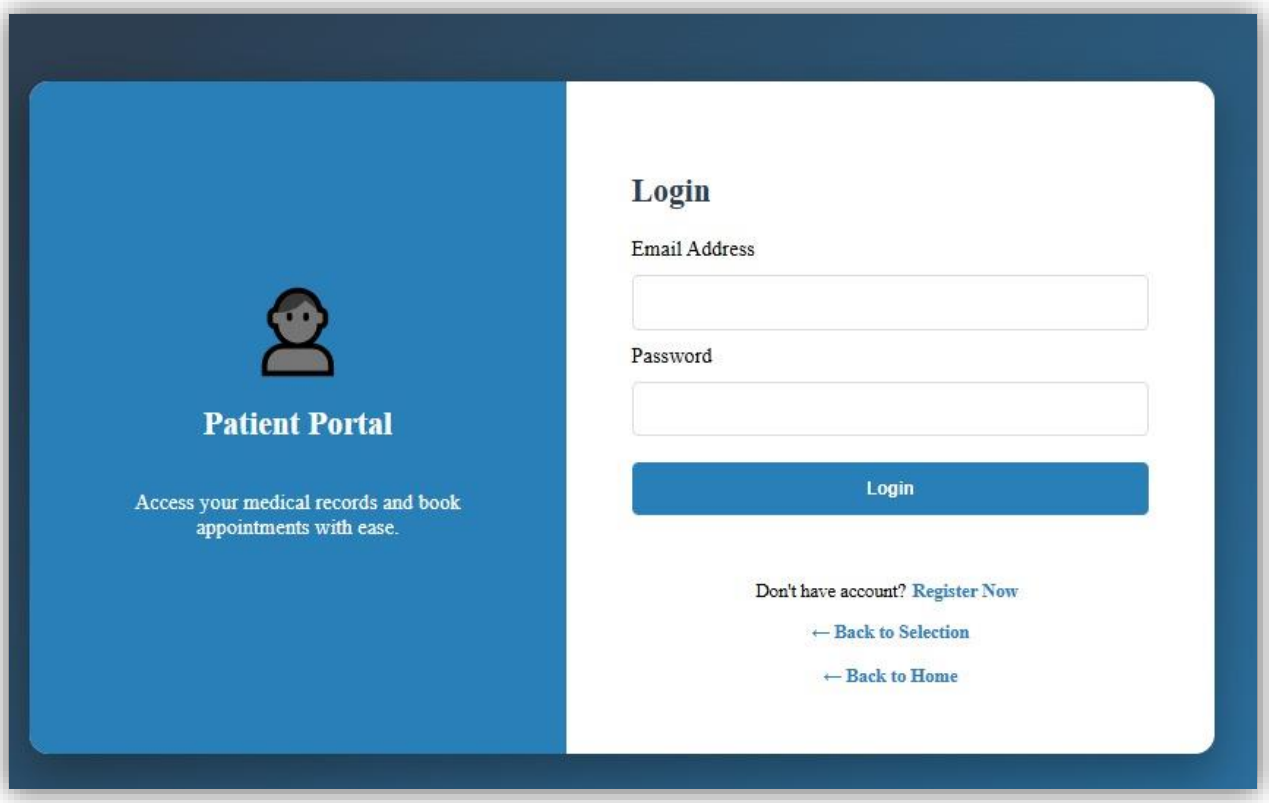
- Split layout (Info + Form)
- Email and password inputs
- Inline validation and error messages
- Redirect to dashboard on success

CSS Highlights:

- Flexbox split layout
- Responsive design
- Hover effect on login button

JavaScript Highlights:

- On input Validation
- Show the error message



Patient Registration

Key Components:

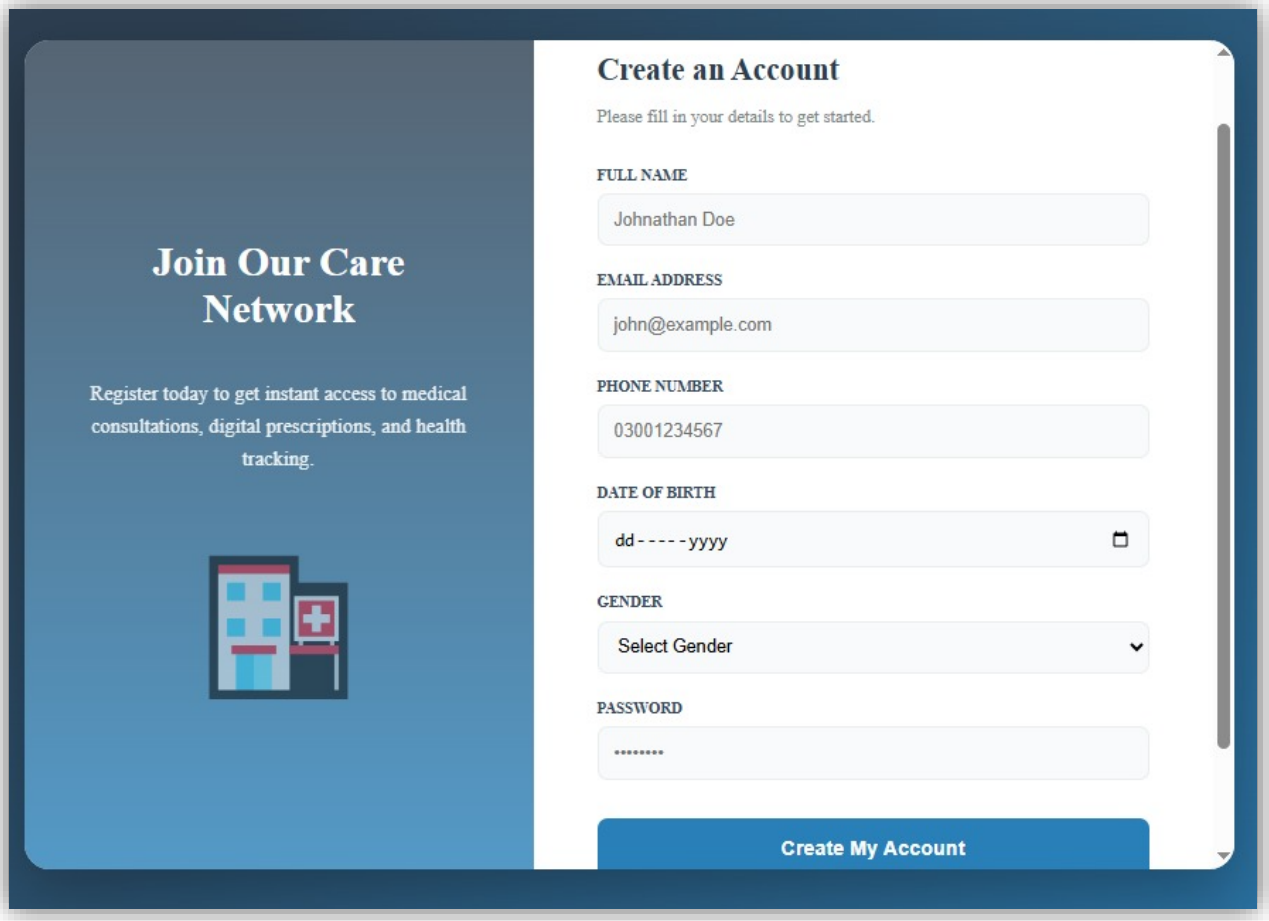
- Grid-based input layout
- Full registration form
- Validation for all fields
- Redirect after successful registration

CSS Highlights:

- Responsive stacking for mobile
- Modern form styling with rounded inputs
- Hover effects on buttons

JavaScript Highlights:

- On input Validation
- Show the error message



Book Appointment Page (Patient)

Key Components:

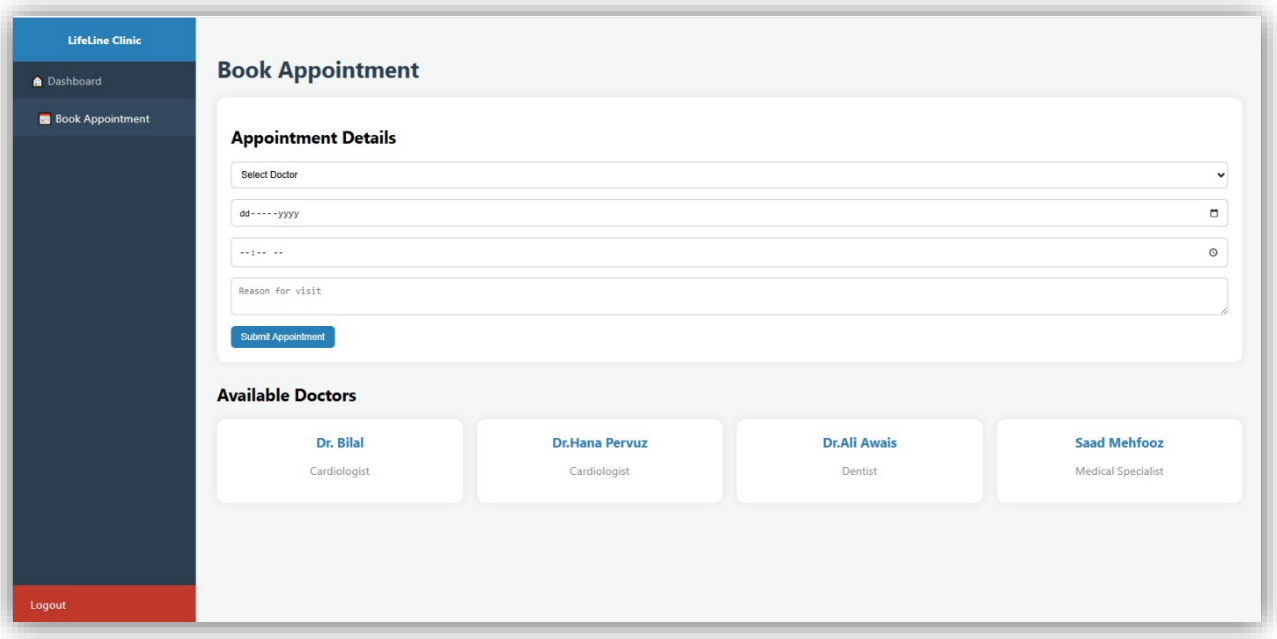
- Sidebar navigation
- Booking form: Doctor selection, Date, Time, Reason
- Doctor cards grid showing available doctors

CSS Highlights:

- Shadowed form cards
- Grid layout for doctor cards
- Hover effects on buttons
- Responsive layout for mobile

JavaScript Highlights:

- On input Validation
- Show the error message
- Manipulation of DOM for list of available Doctors



Patient Dashboard

Key Components:

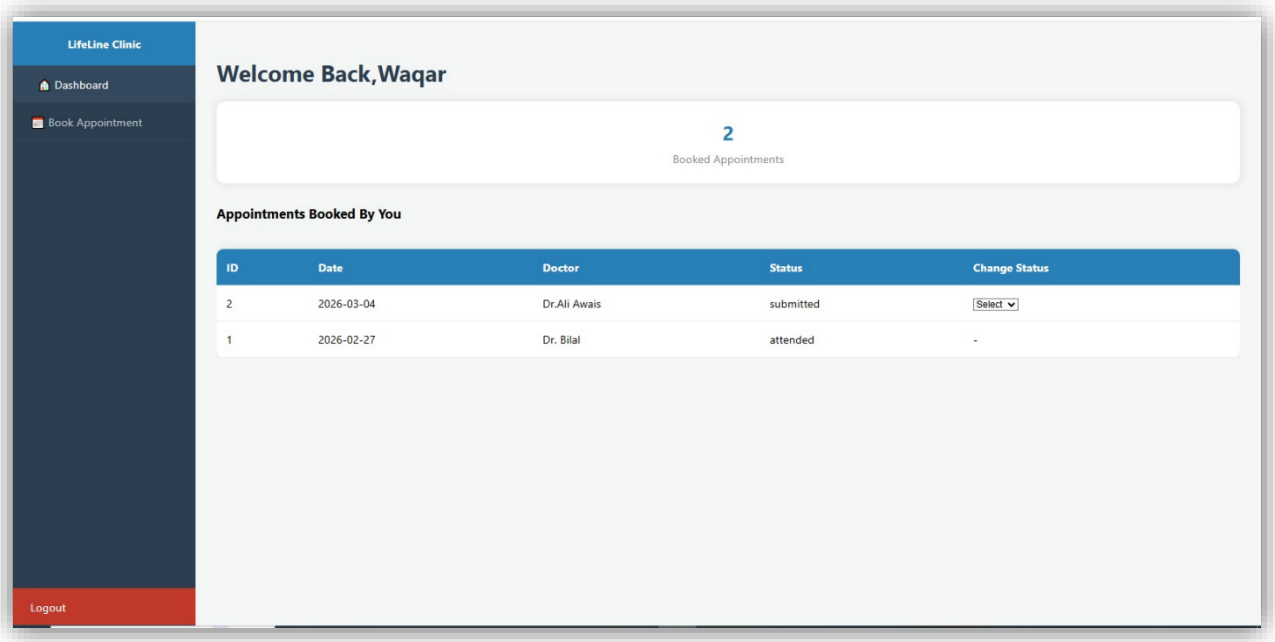
- Sidebar navigation
- Welcome message
- Appointment statistics cards
- Dynamic appointment table
- Appointment status management

CSS Highlights:

- Shadowed stats cards
- Responsive tables and layout
- Mobile-friendly design
- Sidebar toggle functionality

JavaScript Highlights:

- Dynamically showing no. of booked appointments
- Dynamically showing list of booked appointments
- Can cancel the appointment



Admin Login Page

Key Components:

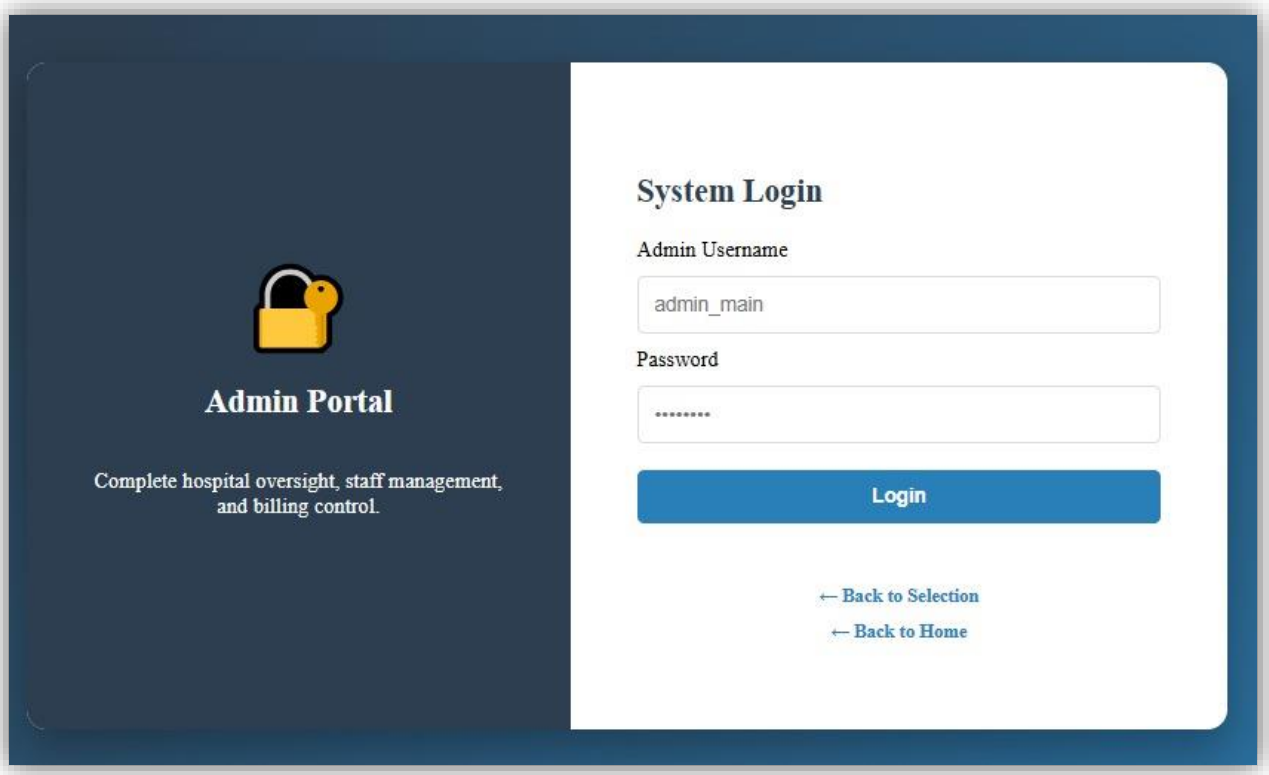
- Split layout (Info + Form)
- Username and password inputs
- Inline validation and error messaging
- Footer links for navigation

CSS Highlights:

- Gradient background
- Flexbox split layout
- Responsive design for tablets and mobile
- Hover effect on login button

JavaScript Highlights:

- On input Validation
- Show the error message



Admin Dashboard

Key Components:

- Sidebar navigation
- Overview stats cards: Registered Doctors, Registered Patients
- Appointments table with delete functionality

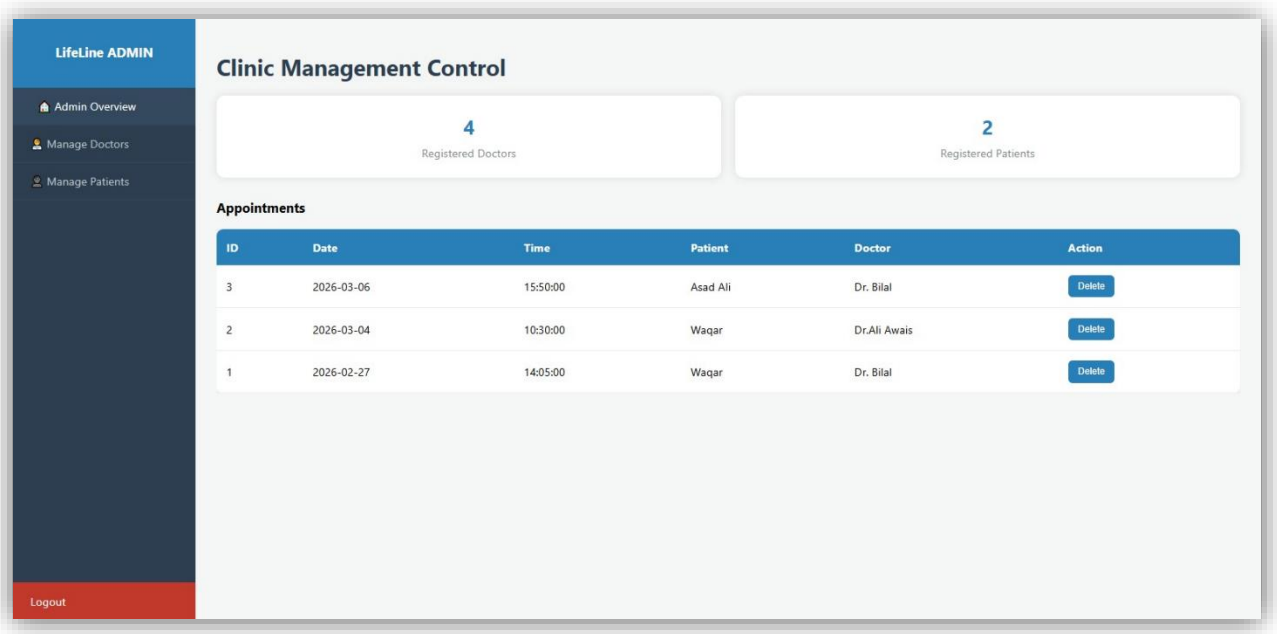
CSS Highlights:

- Shadowed stats cards
- Responsive table
- Sidebar toggle
- Hover effects on action buttons

JavaScript Highlights:

- Dynamically showing no. of registered Doctors
- Dynamically showing no. of registered Patients
- Dynamically showing list of booked appointments
- Handling of Deletion button for each appointments

- Clearing of local storage on pressing of Logout button



Manage Doctors Page (Admin)

Key Components:

- Sidebar navigation
- Add Doctor form (Name, Email, Phone, Specialization, Password)
- Delete Doctor form (by ID)
- Dynamic doctor list table
- Form switch buttons (Add/Delete)
- Logout button

CSS Highlights:

- White form cards with shadow for depth
- Rounded inputs and buttons
- Responsive layout for mobile and tablets
- Sidebar toggling for compact view

JavaScript Highlights:

- Form validation: Name, Email, Phone (11 digits), Specialization, Password (≥4 chars)
- Ensures positive numeric doctor ID for deletion
- Dynamic DOM rendering
- Clearing of local storage on pressing of Logout button

Admin Panel

Admin Overview

Manage Doctors

Manage Patients

Logout

Manage Doctors

Add Doctor | Delete Doctor

Add Doctor

Doctor Name

Email

Specialization

Phone

Set password for doctor

Add Doctor

Doctors List

ID	Name	Email	Specialization	Phone
4	Saad Mehfooz	saad@gmail.com	Medical Specialist	03445332454
3	Dr.Ali Awais	awais@gmail.com	Dentist	03582368901
2	Dr.Hana Pervuz	hanan@gmail.com	Cardiologist	03465612953

Admin Panel

Admin Overview

Manage Doctors

Manage Patients

Logout

Manage Doctors

Add Doctor | Delete Doctor

Delete Doctor

Doctor ID

Delete Doctor

Doctors List

ID	Name	Email	Specialization	Phone
4	Saad Mehfooz	saad@gmail.com	Medical Specialist	03445332454
3	Dr.Ali Awais	awais@gmail.com	Dentist	03582368901
2	Dr.Hana Pervuz	hanan@gmail.com	Cardiologist	03465612953
1	Dr. Bilal	bilal@gmail.com	Cardiologist	03465612953

Manage Patients Page (Admin)

Key Components:

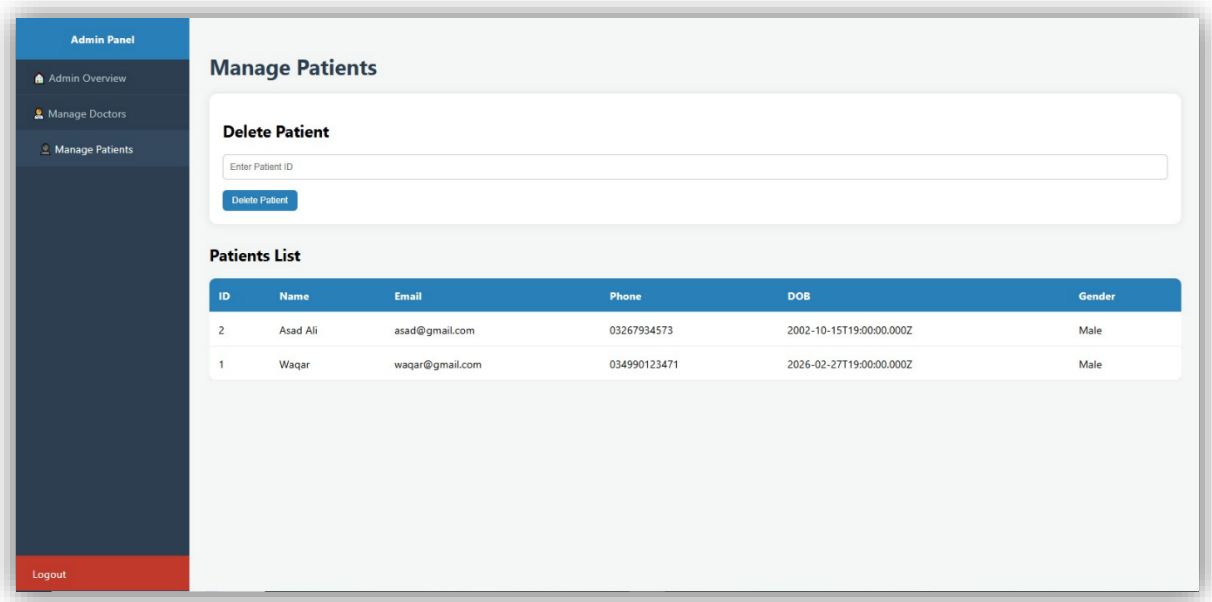
- Sidebar navigation
- Delete patient form
- Dynamic patient table
- Scrollable mobile tables
- Active link highlighting

CSS Highlights:

- Shadowed and rounded form cards
- Responsive table for mobile
- Sidebar toggling and hover effects

JavaScript Highlights:

- No page reload required; dynamic DOM rendering
- Input validation: numeric, positive numbers, non-empty
- Real-time validation listener
- Clears `localStorage` on logout



Doctor Login Page

Key Components:

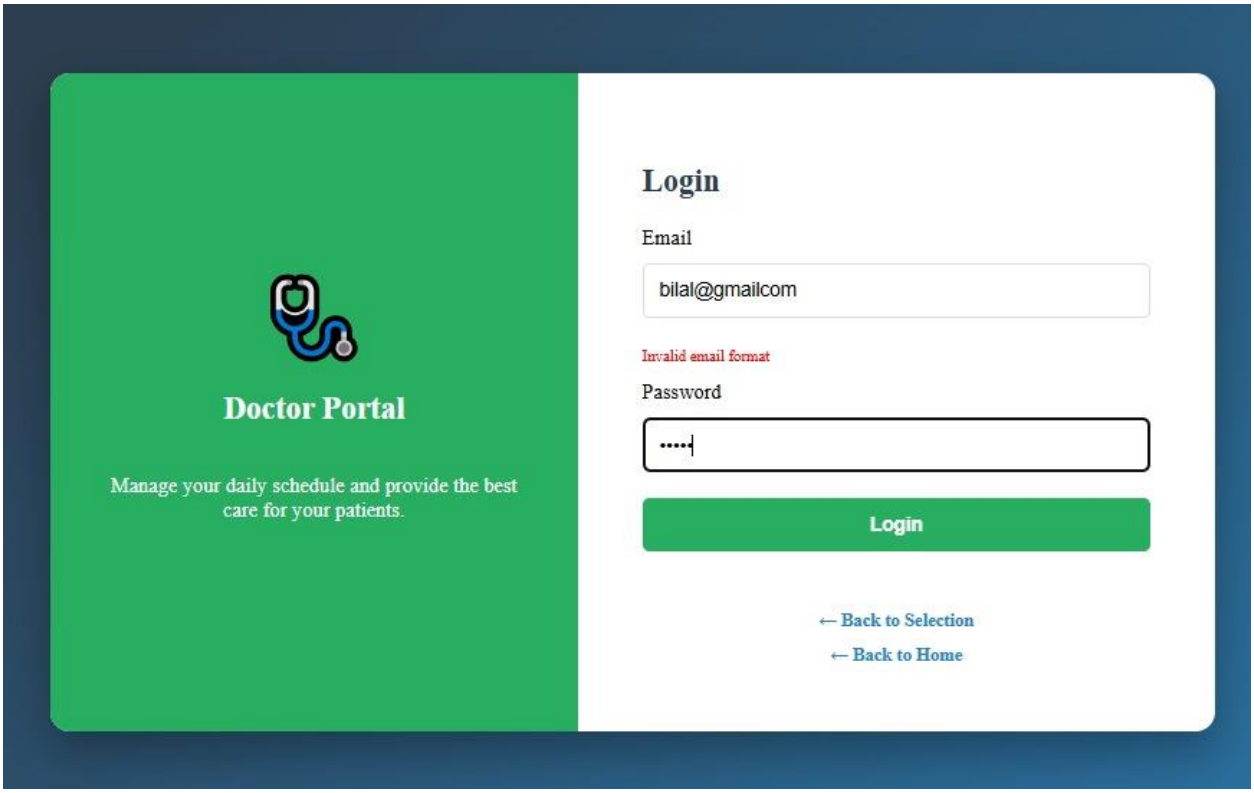
- Split layout (Info + Form)
- Email and password inputs
- Inline validation and error messages
- Footer links for navigation

CSS Highlights:

- Green gradient theme (#27ae60)
- Flexbox split layout
- Responsive design for tablets and mobile
- Hover effect on login button

JavaScript Highlights:

- Validation: email regex, password non-empty
- Real-time inline validation and submit-time checks



Doctor Dashboard

Key Components:

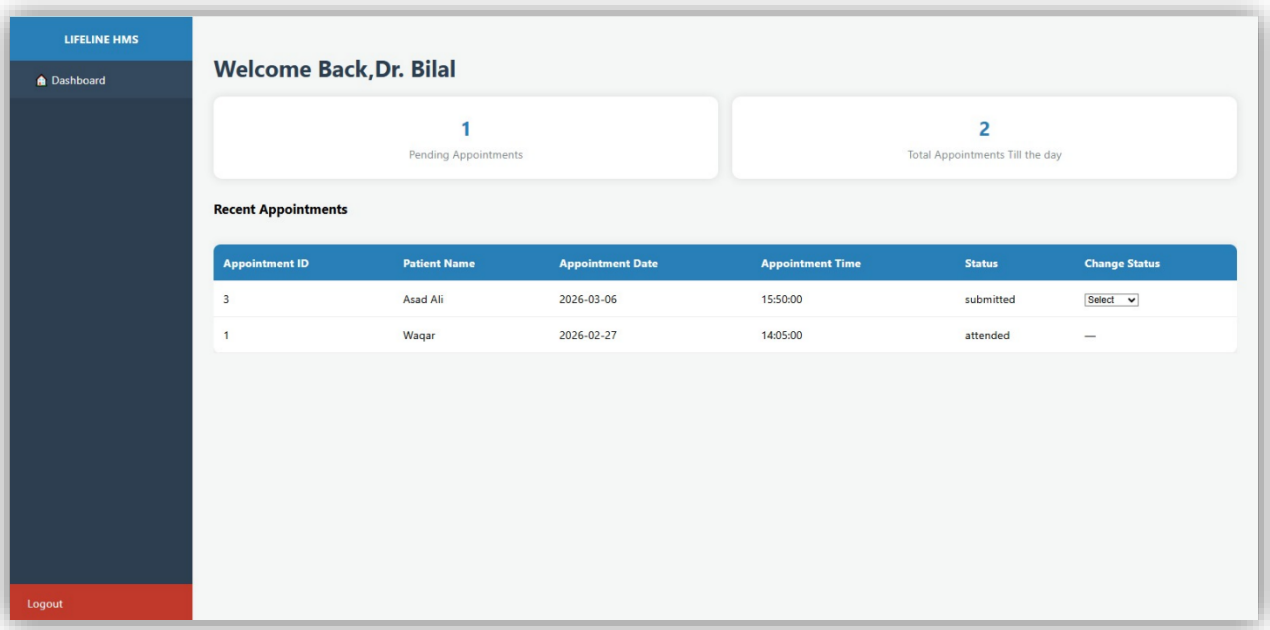
- Sidebar navigation
- Welcome message with doctor name
- Stats cards: Pending and Total Appointments
- Appointment table with status dropdown

CSS Highlights:

- Shadowed stats cards
- Responsive table layout
- Sidebar toggle
- Mobile-friendly design

JavaScript Highlights:

- Dynamically showing no. of pending appointments
- Dynamically showing no. of total appointments
- Dynamically showing list of booked appointments
- Clearing the local storage on logout



Backend Architecture

Modules, Middleware, Routes

JS server.js > ...

```javascript
const express = require('express');
const mysql = require('mysql2');
const bodyParser = require('body-parser');
const path = require('path');

const app = express();

// Middleware
// app.use(bodyParser.urlencoded({ extended: false }));
app.use(express.static('public'));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// home page
app.get('/', (req, res) => {
    res.sendFile(path.join(__dirname, 'home.html'));
});

// page for patients login
app.get('/patientlogin', (req, res) => {
    res.sendFile(path.join(__dirname, 'patient_login.html'));
});

// page for patients registration
app.get('/patientreg', (req, res) => {
    res.sendFile(path.join(__dirname, 'patient_registration.html'));
});

// page for selection of login portal
app.get('/loginportals', (req, res) => {
    res.sendFile(path.join(__dirname, 'login_portals.html'));
});

// page for doctor login
app.get('/doctorlogin', (req, res) => {
    res.sendFile(path.join(__dirname, 'doc_login.html'));
});
```

JS server.js > ...

```javascript

// page for admin login
app.get('/adminlogin', (req, res) => {
    res.sendFile(path.join(__dirname, 'admin_login.html'));
});

// patient dashboard page
app.get('/patientdashboard', (req, res) => {
    res.sendFile(path.join(__dirname, 'patient_dashboard.html'));
});

// admin dashboard page
app.get('/admindashboard', (req, res) => {
    res.sendFile(path.join(__dirname, 'admin_dashboard.html'));
});

// admin dashboard page sub page manage doctors
app.get('/managedoctors', (req, res) => {
    res.sendFile(path.join(__dirname, 'manage_doctors.html'));
});

// admin dashboard sub page manage patients
app.get('/managepatients', (req, res) => {
    res.sendFile(path.join(__dirname, 'manage_patients.html'));
});

// Book Appointment Page
app.get('/bookappointment', (req, res) => {
    res.sendFile(path.join(__dirname, 'book_appointment.html'));
});

// Doctor Dashboard Page
app.get('/doctordashboard', (req, res) => {
    res.sendFile(path.join(__dirname, 'doctor_dashboard.html'));
});
```

Patient APIs

Method	Endpoint	Description
POST	/api/patients/register	Register patient
POST	/api/patients/login	Login
GET	/api/patient/appointments/:id	Fetch appointments
POST	/api/patient/book-appointment	Book appointment
PUT	/api/patient/appointments/:id/status	Cancel appointment
GET	/api/patient/doctors	List doctors

```
// Patient Registration API
app.post('/api/patients/register', (req, res) => {

    const { p_name, p_email, p_phone, p_dob, p_gender, p_pass } = req.body;

    const sql = `INSERT INTO patients (p_name, p_email, p_phone, p_dob, p_gender, p_pass) VALUES (?, ?, ?, ?, ?, ?)`;

    conn.query(sql, [p_name, p_email, p_phone, p_dob, p_gender, p_pass], (err, result) => {

        if (err) {

            if (err.code === 'ER_DUP_ENTRY') {
                return res.json({
                    success: false,
                    message: "Email already exists"
                });
            }

            return res.status(500).json({
                success: false,
                message: "Database error"
            });
        }

        res.json({
            success: true,
            message: "Registration successful"
        });

    }
    );

});
```

```
// Patient Login API
app.post('/api/patients/login', (req, res) => {

    const { email, password } = req.body;

    const sql = "SELECT * FROM patients WHERE p_email = ? AND p_pass = ?";

    conn.query(sql, [email, password], (err, result) => {

        if (err) {
            return res.status(500).json({
                success: false,
                message: "Database error"
            });
        }

        if (result.length > 0) {

            res.json({
                success: true,
                message: "Login successful",
                patient: result[0]
            });

        } else {

            res.json({
                success: false,
                message: "Incorrect email or password"
            });

        }

    });

});
```

```
// api to get all appointments to show patient dashboard
app.get('/api/patient/appointments/:id', (req, res) => {

    const patientId = req.params.id;

    const sql = `
        SELECT
            a.appointment_id,
            a.appointment_date,
            a.status,
            d.d_name
        FROM appointments a
        JOIN doctors d ON a.doctor_id = d.doctor_id
        WHERE a.patient_id = ?
        ORDER BY a.appointment_date DESC
    `;

    conn.query(sql, [patientId], (err, result) => {

        if (err) return res.status(500).json({ success: false });

        res.json(result);
    });
});
```

```
// Book Appointment api
app.post('/api/patient/book-appointment', (req, res) => {

    const { patient_id, doctor_id, appointment_date, appointment_time, reason } = req.body;

    if (!patient_id || !doctor_id || !appointment_date || !appointment_time) {
        return res.json({
            success: false,
            message: "All fields are required"
        });
    }

    const sql = `
        INSERT INTO appointments
        (patient_id, doctor_id, appointment_date, appointment_time, reason_for_visit)
        VALUES (?, ?, ?, ?, ?)
    `;

    conn.query(sql,
        [patient_id, doctor_id, appointment_date, appointment_time, reason],
        (err, result) => {

            if (err) return res.status(500).json({ success: false });

            res.json({
                success: true,
                message: "Appointment submitted successfully"
            });

        });

});
```

```
// api to cancel the appointment if patient want to
app.put('/api/patient/appointments/:id/status', (req, res) => {

    const appointmentId = req.params.id;
    const { status } = req.body;

    if (status !== "cancel") {
        return res.json({
            success: false,
            message: "Invalid status change"
        });
    }

    const sql = "UPDATE appointments SET status = ? WHERE appointment_id = ?";

    conn.query(sql, [status, appointmentId], (err, result) => {

        if (err) return res.status(500).json({ success: false });

        res.json({
            success: true,
            message: "Appointment cancelled"
        });

    });

});
```

```
// Get doctors for booking appointment , showing on patient dashboard
app.get('/api/patient/doctors', (req, res) => {

    const sql = "SELECT doctor_id, d_name, d_specialization FROM doctors";

    conn.query(sql, (err, result) => {

        if (err) return res.status(500).json({ success: false });

        res.json(result);

    });

});
```

Doctor APIs

Method	Endpoint	Description
POST	/api/doctors/login	Login
GET	/api/doctor/dashboard/:doctorId	Dashboard
PUT	/api/appoint tments/:id	Update status

```
//api for doctor login
app.post('/api/doctors/login', (req, res) => {

    const { email, password } = req.body;

    const sql = "SELECT * FROM doctors WHERE d_email = ? AND d_pass = ?";

    conn.query(sql, [email, password], (err, result) => {
        if (err) {
            return res.status(500).json({
                success: false,
                message: "Database error"
            }); }
        if (result.length > 0) {
            res.json({
                success: true,
                message: "Login successful",
                doctor: result[0]
            }); }
        else {
            res.json({
             success: false,
             message: "Incorrect email or password"
            }); }
    });
});
```

```
// api that retrive data for doctor dashboard
app.get("/api/doctor/dashboard/:doctorId", (req, res) => {

    const doctorId = req.params.doctorId;

    const sql = `
        SELECT
            a.appointment_id,
            a.appointment_date,
            a.appointment_time,
            a.status,
            p.p_name
        FROM appointments a
        JOIN patients p ON a.patient_id = p.patient_id
        WHERE a.doctor_id = ?
        ORDER BY a.appointment_date DESC
    `;

    conn.query(sql, [doctorId], (err, results) => {

        if (err) {
            return res.status(500).json({ message: "Database error" });
        }

        res.json(results);

    });

});
```

```
// api to update the status changed by doctor
app.put("/api/appointments/:id", (req, res) => {

    const appointmentId = req.params.id;
    const { status } = req.body;

    const sql = `
        UPDATE appointments
        SET status = ?
        WHERE appointment_id = ?
    `;

    conn.query(sql, [status, appointmentId], (err) => {

        if (err) {
            return res.status(500).json({ message: "Update failed" });
        }

        res.json({ message: "Status updated" });

    });

});
```

Admin APIs

Method	Endpoint	Description
POST	/api/admin/login	Login
GET	/api/admin/dashboard-stats	Stats
GET	/api/admin/appointments	List appointments
DELETE	/api/admin/appointments/:id	Delete appointment
GET	/api/admin/doctors	List doctors
POST	/api/admin/doctors	Add doctor
DELETE	/api/admin/doctors/:id	Delete doctor
GET	/api/admin/patients	List patients
DELETE	/api/admin/patients/:id	Delete patient

```
// api for admin login
// Admin Login API
app.post('/api/admin/login', (req, res) => {

    const { email, password } = req.body;

    const sql = "SELECT * FROM admins WHERE admin_username = ? AND admin_password = ?";

    conn.query(sql, [email, password], (err, result) => {

        if (err) {
            return res.status(500).json({
                success: false,
                message: "Database error"
            });
        }

        if (result.length > 0) {

            res.json({
                success: true,
                message: "Login successful",
                admin: result[0]
            });

        } else {

            res.json({
                success: false,
                message: "Incorrect username or password"
            });

        }

    });

});
```

```
// api run on admin dashboard page load
app.get('/api/admin/dashboard-stats', (req, res) => {

    const doctorQuery = "SELECT COUNT(*) AS totalDoctors FROM doctors";
    const patientQuery = "SELECT COUNT(*) AS totalPatients FROM patients";

    conn.query(doctorQuery, (err1, doctorResult) => {

        if (err1) return res.status(500).json({ success: false });

        conn.query(patientQuery, (err2, patientResult) => {

            if (err2) return res.status(500).json({ success: false });

            res.json({
                totalDoctors: doctorResult[0].totalDoctors,
                totalPatients: patientResult[0].totalPatients
            });

        });

    });

});
```

```
// api run on admin dashboard page load for list of appointments
app.get('/api/admin/appointments', (req, res) => {

    const sql = `
        SELECT
            a.appointment_id,
            a.appointment_date,
            a.appointment_time,
            p.p_name,
            d.d_name
        FROM appointments a
        JOIN patients p ON a.patient_id = p.patient_id
        JOIN doctors d ON a.doctor_id = d.doctor_id
        ORDER BY a.appointment_date DESC
    `;

    conn.query(sql, (err, result) => {

        if (err) {
            return res.status(500).json({ success: false });
        }

        res.json(result);

    });

});
```

```
// this api is for if admin want to delete an appointment
app.delete('/api/admin/appointments/:id', (req, res) => {

    const appointmentId = req.params.id;

    const sql = "DELETE FROM appointments WHERE appointment_id = ?";

    conn.query(sql, [appointmentId], (err, result) => {

        if (err) {
            return res.status(500).json({ success: false });
        }

        res.json({
            success: true,
            message: "Appointment deleted"
        });

    });

});
```

```
// Get All Doctors
app.get('/api/admin/doctors', (req, res) => {

    const sql = "SELECT * FROM doctors ORDER BY doctor_id DESC";

    conn.query(sql, (err, result) => {

        if (err) {
            return res.status(500).json({
                success: false,
                message: "Database error"
            });
        }

        res.json(result);

    });

});
```

```
// Add Doctor
app.post('/api/admin/doctors', (req, res) => {

    const { d_name, d_email, d_phone, d_specialization, d_pass } = req.body;

    if (!d_name || !d_email || !d_phone || !d_specialization || !d_pass) {
        return res.json({
            success: false,
            message: "All fields are required"
        });
    }

    const sql = `
        INSERT INTO doctors (d_name, d_email, d_phone, d_specialization,d_pass) VALUES (?, ?, ?, ?, ?) `;

    conn.query(sql, [d_name, d_email, d_phone, d_specialization, d_pass], (err, result) => {

        if (err) {

            if (err.code === 'ER_DUP_ENTRY') {
                return res.json({
                    success: false,
                    message: "Email already exists"
                });
            }

            return res.status(500).json({
                success: false,
                message: "Database error"
            });
        }

        res.json({
            success: true,
            message: "Doctor added successfully"
        });

    });
```

```
// Delete Doctor
app.delete('/api/admin/doctors/:id', (req, res) => {

    const doctorId = req.params.id;

    const sql = "DELETE FROM doctors WHERE doctor_id = ?";

    conn.query(sql, [doctorId], (err, result) => {

        if (err) {
            return res.status(500).json({
                success: false,
                message: "Database error"
            });
        }

        if (result.affectedRows === 0) {
            return res.json({
                success: false,
                message: "Doctor not found"
            });
        }

        res.json({
            success: true,
            message: "Doctor deleted successfully"
        });

    });

});
```

```
// Get All Patients
app.get('/api/admin/patients', (req, res) => {

    const sql = "SELECT * FROM patients ORDER BY patient_id DESC";

    conn.query(sql, (err, result) => {

        if (err) {
            return res.status(500).json({
                success: false,
                message: "Database error"
            });
        }

        res.json(result);

    });

});
```

```
// Delete Patient
app.delete('/api/admin/patients/:id', (req, res) => {

    const patientId = req.params.id;

    const sql = "DELETE FROM patients WHERE patient_id = ?";

    conn.query(sql, [patientId], (err, result) => {

        if (err) {
            return res.status(500).json({
                success: false,
                message: "Database error"
            });
        }

        if (result.affectedRows === 0) {
            return res.json({
                success: false,
                message: "Patient not found"
            });
        }

        res.json({
            success: true,
            message: "Patient deleted successfully"
        });

    });

});
```

Database Design

Database Name

clinicManagement

Database Overview

The database is designed using **MySQL** and follows the **Relational Database Model**.
It consists of four main tables:

- Patients
- Doctors
- Appointments
- Admins

The database ensures:

- Data integrity using **Primary Keys & Foreign Keys**
- Relationship consistency using **ON DELETE CASCADE**
- Data validation using **NOT NULL, UNIQUE, ENUM constraints**

Entity Relationship Overview

Entities:

1. Patient
2. Doctor
3. Appointment
4. Admin

Relationships:

- One Patient → Many Appointments
- One Doctor → Many Appointments
- One Appointment → Belongs to One Patient & One Doctor

This represents a **Many-to-Many relationship** between Patients and Doctors resolved through the **Appointments** table.

Patients Table

Purpose:
Stores patient personal and login information.

Field Name	Data Type	Description
patient_id	INT (PK)	Unique ID for each patient
p_name	VARCHAR(100)	Patient full name
p_email	VARCHAR(100)	Unique email address
p_phone	VARCHAR(20)	Contact number
p_dob	DATE	Date of birth
p_geneder	VARCHAR(10)	Gender
p_pass	VARCHAR(255)	Encrypted password
created_at	TIMESTAMP	Record creation time

Constraints:

- `patient_id` → Primary Key
- `p_email` → Unique
- Required fields → NOT NULL

```
function createTables() {

    // Patients Table
    const patientsTable = `
    CREATE TABLE patients (
        patient_id INT AUTO_INCREMENT PRIMARY KEY,
        p_name VARCHAR(100) NOT NULL,
        p_email VARCHAR(100) UNIQUE NOT NULL,
        p_phone VARCHAR(20) NOT NULL,
        p_dob DATE NOT NULL,
        p_geneder VARCHAR(10) NOT NULL,
        p_pass VARCHAR(255) NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )`;
```

Doctors Table

Purpose:
Stores doctor details and login credentials.

Field Name	Data Type	Description
doctor_id	INT (PK)	Unique ID for each doctor
d_name	VARCHAR(100)	Doctor full name
d_email	VARCHAR(100)	Unique email
d_phone	VARCHAR(20)	Contact number
d_specialization	VARCHAR(100)	Medical specialization
d_pass	VARCHAR(255)	Encrypted password
created_at	TIMESTAMP	Record creation time

Constraints:

- `doctor_id` → Primary Key
- `d_email` → Unique
- Required fields → NOT NULL

```
// Doctors Table
const doctorsTable = `
CREATE TABLE doctors (
    doctor_id INT AUTO_INCREMENT PRIMARY KEY,
    d_name VARCHAR(100) NOT NULL,
    d_email VARCHAR(100) NOT NULL UNIQUE,
    d_phone VARCHAR(20),
    d_specialization VARCHAR(100) NOT NULL,
    d_pass VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)`;
```

Appointments Table

Purpose:
Manages appointment bookings between patients and doctors.

Field Name	Data Type	Description
appointment_id	INT (PK)	Unique appointment ID
patient_id	INT (FK)	References Patients table
doctor_id	INT (FK)	References Doctors table
appointment_date	DATE	Appointment date
appointment_time	TIME	Appointment time
status	ENUM	Appointment status
created_at	TIMESTAMP	Record creation time

Status Values:

- submitted
- cancel
- attended

Constraints:

- `appointment_id` → Primary Key
- `patient_id` → Foreign Key → patients(patient_id)
- `doctor_id` → Foreign Key → doctors(doctor_id)
- ON DELETE CASCADE applied

Relationship Explanation:

- One Patient can book multiple appointments.
- One Doctor can have multiple appointments.
- If a Patient or Doctor is deleted, their appointments are automatically deleted.

```
// Appointments Table
const appointmentsTable = `
CREATE TABLE appointments (
    appointment_id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
    status ENUM('submitted','cancel','attended') DEFAULT 'submitted',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id) ON DELETE CASCADE,
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id) ON DELETE CASCADE
)`;
```

Admins Table

Purpose:
Stores system administrator credentials.

Field Name	Data Type	Description
admin_id	INT (PK)	Unique admin ID
admin_username	VARCHAR(50)	Unique username
admin_password	VARCHAR(255)	Encrypted password
created_at	TIMESTAMP	Record creation time

Constraints:

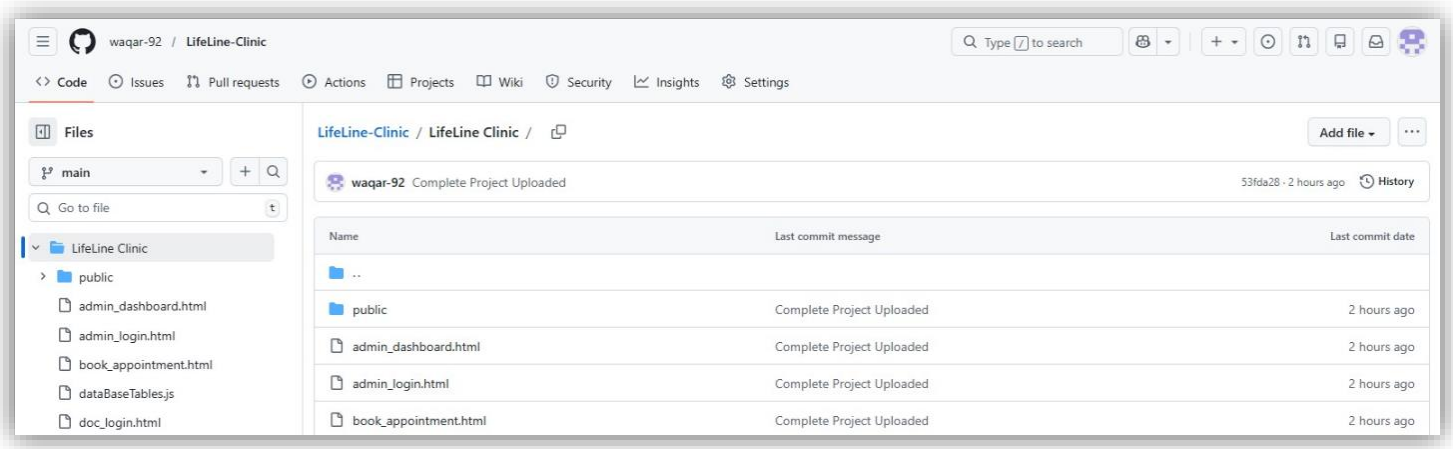
- `admin_id` → Primary Key
- `admin_username` → Unique

```
// Admins Table
const adminsTable = `
CREATE TABLE admins (
    admin_id INT AUTO_INCREMENT PRIMARY KEY,
    admin_username VARCHAR(50) NOT NULL UNIQUE,
    admin_password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)`;
```

Github Project Repository

Link

https://github.com/waqar-92/LifeLine-Clinic



Conclusion & Future Enhancements

Conclusion

The Clinic Management System successfully provides a complete digital solution for managing clinic operations efficiently. The system integrates patients, doctors, and admin into single platform with role-based authentication and secure access control. Patients can register, log in, and book appointments seamlessly, while doctors can manage and update appointment statuses. Administrators have full control over doctors, patients, and appointments through dynamic dashboards and management pages.

The project demonstrates effective use of modern web technologies including frontend development, RESTful APIs, backend logic, and database integration. With real-time validation, dynamic rendering without page reloads, and responsive design, the system ensures a smooth and user-friendly experience across devices. Overall, the project fulfills its objective of automating clinic workflows and reducing manual management processes.

Future Enhancements

- Add **password encryption (bcrypt hashing)** for secure user credential storage.
- Integrate **online payment system** for paid appointment booking.
- Add **Email notification system** for appointment confirmation and reminders.
- Develop a **Medical Records Module** where doctors can upload prescriptions and reports.
- Add **Appointment Calendar View** for better scheduling visualization.
- Enable **PDF and Excel report export** functionality.
- Add **search and filter options** in all management tables.
- Add **automatic database backup and recovery system**.
- Add functionality of **modify details and change Password**

THE END